From Einstein-Boltzmann Equations to Galaxy Power Spectra: Physicist-Supervised, Oracle-Guided AI Reimplementation in JAX

Anonymous Authors[1]

Abstract

Gradient-based inference methods such as Hamiltonian Monte Carlo require differentiable forward models, and simulation-based inference benefits from them, yet no existing code provides full CMB spectra plus one-loop galaxy power spectra with exact automatic differentiation on GPU. We address this gap with CLAX and CLAX-PT, fully differentiable JAX reimplementations of the CLASS Boltzmann solver and its one-loop perturbation theory extension CLASS-PT, developed via physicist-supervised, oracle-guided AI agents that iterate against reference data from CLASS and CLASS-PT, autonomously resolving implementation bugs and escalating to human review when oracle feedback stalls. Across the development, agents autonomously resolved 38 of 40 documented bugs (95%) without human input. The remaining 5% required human physics judgment and fell into exactly two failure modes: *architectural incoherence* (code structure incompatible with the target physics) and *oracle-passing fudge factors* (a numerical patch satisfying all tests but encoding no physical model). CLAX achieves $< 0.2\%$ lensed CMB accuracy at $\ell \leq 2000$ and $< 0.6\%$ unlensed C_ℓ^{TT} at $\ell \leq 1000$; CLAX-PT achieves $< 0.7\%$ galaxy power spectrum accuracy for monopole and quadrupole spectra and $< 1.5\%$ for hexadecapoles at $k < 0.3\ h/\mathrm{Mpc}$. Automatic differentiation gradients agree with fourth-order finite differences to 0.15% on average. The full pipeline runs at 34 s/step on a single V100 GPU with HMC-ready accuracy. We characterize the boundary between unsupervised and supervised modes of AI-assisted scientific development, propose three principles for structuring human–agent collaboration, and compare against concurrent differentiable cosmology codes DISCO-EB and ABCMB.

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the AI4Physics Workshop at the International Conference on Machine Learning (AI4Physics@ICML 2026). Do not distribute.

1. Introduction

The standard model of cosmology is tested by comparing theoretical predictions against observations from galaxy surveys such as DESI (DESI Collaboration, 2025), Euclid, and CMB experiments including Planck. This comparison increasingly benefits from gradient-based inference: exact Fisher matrices require derivatives $\partial C_\ell / \partial \theta_i$, HMC sampling (Neal, 2011) requires $\nabla_\theta \log P(\boldsymbol{\theta}|\mathrm{data})$, and even simulation-based inference (Cranmer et al., 2020)—which does not require gradients—can leverage them for score-matching and summary-network training. Gradient-based inference libraries such as NumPyro (Phan et al., 2019), BlackJAX (Cabezas et al., 2024), and normalizing-flow-augmented MCMC (Gabrié et al., 2022) are mature, but the missing ingredient is a differentiable cosmological forward model that can be passed to them.

The dominant Boltzmann codes—CLASS (Blas et al., 2011) and CAMB (Lewis et al., 2000)—are written in C and Fortran, are not differentiable, and are not GPU-native. Several JAX implementations exist but cover only subsets of the problem: DISCO-EB (Hahn et al., 2023) solves the linear Einstein–Boltzmann equations to compute the matter power spectrum $P(k)$ but not CMB spectra or galaxy clustering observables; ABCMB (Zhou et al., 2026) implements the CMB pipeline including lensing and massive neutrinos but does not include perturbation theory for galaxy surveys; JAX-COSMO (Lemos et al., 2023) computes the nonlinear matter power spectrum via fitting formulae but does not solve the full Boltzmann hierarchy; and SymBoltz.jl (Sletmoen, 2024) provides a full Boltzmann solver in Julia but without GPU support. No existing code provides the full chain from cosmological parameters through CMB angular power spectra *and* one-loop EFT galaxy power spectrum multipoles, with end-to-end automatic differentiation on GPU.

We describe the development of CLAX and CLAX-PT—fully differentiable JAX reimplementations of the CLASS and CLASS-PT pipelines—and the oracle-guided agentic method-

ology used to build them. Using CLASS v3.3.4 and CLASS-PT (Chudaykin et al., 2021) as ground truth, we find a clear boundary: agents handle implementation bugs (convention errors, algorithmic transcription, coefficient verification) autonomously and reliably, but fail at two classes of problem that require human physics judgment: (1) recognizing that an architecture is structurally incompatible with the intended physics, and (2) detecting that a numerically effective patch satisfies all oracle tests at the calibration point while encoding incorrect physics.

Contributions.

- CLAX: a JAX reimplementation of the full CLASS CMB pipeline (background, thermodynamics, perturbations, transfer, lensing), achieving $< 0.1\%$ accuracy at $\ell \leq 2000$ with JIT-compiled GPU execution at 34 s/step (V100, `fit_cl` preset).
- CLAX-PT: a JAX reimplementation of CLASS-PT's one-loop EFT power spectra with IR resummation and RSD multipoles, achieving $< 0.7\%$ accuracy for monopole and quadrupole spectra.
- Validated autodifferentiation gradients matching finite differences to 0.15% across cosmological parameters.
- A landscape comparison against DISCO-EB, ABCMB, CLASS, and CLASS-PT, showing that CLAX is the first code combining CMB and one-loop galaxy spectra with full AD on GPU.
- An empirical bug taxonomy of 40 bugs across the development, classifying autonomous vs. human-required resolution and identifying two specific failure modes of oracle-guided development.

2. Background

2.1. Cosmological inference and differentiable Boltzmann codes

Galaxy surveys and CMB experiments measure the angular power spectrum C_ℓ and the galaxy power spectrum $P_{\rm gg}(k, \ell)$, which depend on cosmological parameters $\boldsymbol{\theta}$ (density, dark matter density, Hubble constant, spectral index, etc.) through the Boltzmann equation—a coupled system of ODEs for photon, baryon, dark matter, and neutrino perturbations that must be integrated numerically. Both are expensive to approximate by finite differences and practical to compute exactly only with automatic differentiation (AD) for codes of this complexity. While concurrent work (ABCMB, SymBoltz.jl) provides AD for CMB spectra, CLAX is the first code to combine differentiable CMB C_ℓ with one-loop galaxy power spectrum multipoles, enabling `jax.grad` through the full cosmological analysis pipeline for galaxy clustering.

2.2. AI-assisted scientific code development

The oracle-based methodology we employ is inspired by the AI C compiler project (Carlini, 2026), in which parallel Claude Code agents produced a Rust C compiler using GCC as an oracle. Key infrastructure includes: (i) a shared `CHANGELOG.md` updated after every session to prevent re-exploration of dead ends; (ii) test-first development where every module's tests are written against reference data before implementation; (iii) a `--fast` flag sampling 10% of test cases to prevent "time blindness"; and (iv) parallel agent teams using git worktrees to explore competing hypotheses independently.

The critical challenge not addressed in the compiler setting is *domain knowledge*: CLASS is not just a large codebase—it encodes decades of cosmological physics knowledge, conventions, and approximations that require physics expertise to interpret. This makes the boundary between supervised and unsupervised development especially important to characterize.

3. Methodology

3.1. Pipeline architecture

CLAX implements a sequential pipeline of pure functions, each returning a frozen PyTree:

$$\text{CosmoParams} \to \text{BG} \to \text{TH} \to \text{EB} \to \text{PR} \to \text{TR} \to \text{HR} \to \text{LE}$$

where BG is background cosmology, TH is thermodynamics/recombination, EB is the Einstein–Boltzmann perturbation hierarchy, PR is the primordial spectrum, TR is transfer functions, HR is harmonic (C_ℓ) integration, and LE is CMB lensing via Wigner d-functions. `CosmoParams` fields are JAX-traced (for AD); `PrecisionParams` fields are static (control array shapes). No branching on traced values ensures compatibility with `jax.jit` and `jax.grad`.

The Einstein–Boltzmann system is integrated using Kvaerno5 (ESDIRK, default) or Rodas5 (Rosenbrock) solvers from the Diffrax library (Kidger, 2021), with a DISCO-EB-style filtered PID step controller using k-dependent weighting on six key perturbation variables. Reverse-mode AD through the ODE solve uses `RecursiveCheckpointAdjoint` for memory efficiency, with `custom_vjp` wrappers for the shooting method ($\theta_s \to H_0$ via implicit differentiation) and the Saha equilibrium (implicit-function-theorem JVP).

CLAX-PT extends the pipeline with one-loop EFT power spectra following the CLASS-PT algorithm: FFTLog decomposition of P_{22} and P_{13} integrals (Fang et al., 2017),

precomputed M_{22}/M_{13} kernel matrices, IR resummation via DST with odd/even spline mode removal (Senatore & Zaldarriaga, 2015; Vlah et al., 2015), an EFT bias expansion ($b_1, b_2, b_{s^2}, b_{3\mathrm{nl}}$, counterterms, stochastic contributions), and RSD multipoles ($\ell = 0, 2, 4$) via Gauss–Legendre quadrature over μ with anisotropic BAO damping. The entire CLAX-PT module (∼2,100 lines) is end-to-end differentiable: `jax.grad` flows through FFTLog, IR resummation, bias assembly, and GL quadrature.

3.2. Development methodology

The CLAX CMB pipeline (∼10,000 lines) was developed over ∼6 days across ≥100 Claude Code (Opus 4.6) agent sessions. The CLAX-PT one-loop PT module (∼2,100 lines) was developed over 12 days and 57 logged sessions. Both phases used the same oracle-guided methodology: every module has a test file written *before* implementation, specifying what CLASS (or CLASS-PT) produces; the code is finished when and only when the oracle tests pass.

The supervision methodology—encoded in `CLAUDE.md` and refined across both development phases—proved essential. Two rules were particularly important: (1) never add fudge factors (if a test fails at 0.2%, find the actual bug), and (2) test at multiple parameter points, not just the fiducial cosmology. Both rules directly prevented specific failure modes documented in Section 5.

4. Results: Accuracy and Performance

4.1. Landscape comparison

Table 1 compares CLAX against existing differentiable and non-differentiable cosmology codes. CLAX is the first JAX code providing both CMB angular power spectra and one-loop galaxy power spectrum multipoles with end-to-end automatic differentiation on GPU.

4.2. CMB pipeline accuracy

Table 2 summarizes accuracy of CLAX against CLASS at the Planck 2018 best-fit cosmology (`planck_cl` preset). Background quantities match to $< 0.01\%$; thermodynamics to $< 0.1\%$. CMB temperature and polarization spectra are within 0.02%–0.57% across $\ell = 20$–1000. Lensed spectra at $\ell = 2000$ pass at $< 0.2\%$. Multi-cosmology validation across 10 parameter points (omega_b/cdm $\pm 20\%$, $h \pm 10\%$, massive neutrinos 0.06–0.3 eV, $N_{\mathrm{eff}} = 2.0$–4.0) confirms sub-0.5% accuracy throughout parameter space.

4.3. Galaxy power spectrum accuracy

Table 3 shows accuracy for one-loop EFT galaxy power spectra at $z = 0.38$ (BOSS effective redshift) against CLASS-PT. Real-space spectra (P_{mm}, P_{gg}, P_{gm}) agree to $< 0.31\%$ (mean 0.04%). RSD monopoles ($\ell = 0$) agree to $< 0.6\%$ for matter and galaxies. Quadrupoles ($\ell = 2$) agree to $< 0.7\%$ for matter and $< 0.9\%$ for galaxies. Hexadecapoles ($\ell = 4$) are validated using $|\Delta| / \max(|\mathrm{ref}|) < 2\%$ since CLASS-PT values are small and the spectrum crosses near zero.

4.4. Gradient verification

Automatic differentiation gradients (reverse-mode, using `jax.grad`) agree with fourth-order centered finite differences to 0.15% on average, with 97.7% of k-modes passing a 1% threshold. Table 4 shows the parameter-by-parameter breakdown for the matter power spectrum $P_{mm}(k)$ at $k = 0.1\ h/\mathrm{Mpc}$.

Key technical ingredients enabling stable AD through the full pipeline include: (1) `RecursiveCheckpointAdjoint` for memory-efficient reverse-mode through the stiff perturbation ODE; (2) a `custom_vjp` on the shooting method ($\theta_s \to H_0$) implementing implicit differentiation; and (3) a `custom_jvp` on the hydrogen Saha equilibrium equation, replacing explicit `stop_gradient` calls with implicit-function-theorem tangents to restore nonzero sensitivities around recombination.

4.5. Performance

CLAX offers two presets for different use cases (Table 5). The `fit_cl` preset uses 20 k/decade, $\ell_{\max} = 17$ hierarchy, table-based Bessel functions, and `rtol` $= 10^{-3}$ to achieve 34 s/step on a V100 GPU—sufficient for HMC. The `planck_cl` preset uses 60 k/decade, $\ell_{\max} = 50$, and tight tolerances for science-grade accuracy at ∼487 s on H100. An alternative Rodas5 Rosenbrock solver (Di Marzo, 1993), following DISCO-EB, avoids Newton iteration entirely (one LU factorization per step instead of iterative convergence). A batched variant (`Rodas5Batched`) solves all k-modes in a single `diffeqsolve` call with shared adaptive time-stepping across the batch, vmapping the Jacobian computation and LU factorization—the same architecture used by DISCO-EB. On CPU, the unbatched Rodas5 yields a measured ∼1.3× speedup over Kvaerno5; the batched variant is expected to offer substantially larger gains on GPU by saturating GPU throughput with large batched linear algebra.

4.6. Application: gradient scaling

The primary advantage of AD over finite differences for cosmological inference is the scaling of gradient cost with the number of parameters d. Finite-difference gradients require $2d$ forward evaluations (centered differences); reverse-mode AD requires one forward pass plus one backward pass regardless of d. The backward pass typically costs 2–4× the

Table 1. Comparison of cosmological theory codes. "AD" = automatic differentiation. "PT" = one-loop perturbation theory for galaxy clustering. CLAX is the only code combining CMB C_ℓ, lensing, $P(k)$, and 1-loop PT with full AD on GPU.

Code	Framework	C_ℓ	Lensing	$P(k)$	1-loop PT	AD	GPU
CLASS + CLASS-PT	C / Python	✓	✓	✓	✓	—	—
CAMB	Fortran	✓	✓	✓	—	—	—
DISCO-EB	JAX	—	—	✓	—	✓	✓
ABCMB	JAX	✓	✓	✓	—	✓	✓
SymBoltz.jl	Julia	✓	—	✓	—	✓	—
CLAX + CLAX-PT	JAX	✓	✓	✓	✓	✓	✓

Table 2. CMB accuracy (CLAX vs. CLASS, `planck_cl` preset, Planck 2018 fiducial).

Quantity	Max error
$H(z)$, $D_A(z)$, r_s	$< 0.01\%$
$x_e(z)$, $g(\tau)$	$< 0.1\%$
C_ℓ^{TT}, $\ell = 20, 100, 500, 1000$	0.08%, 0.02%, 0.15%, 0.57%
C_ℓ^{EE}, $\ell = 20, 100, 500, 1000$	0.21%, 0.02%, 0.15%, 0.26%
Lensed C_ℓ, $\ell = 2000$	$< 0.2\%$
Multi-cosmology (10 pts)	$< 0.5\%$

Table 3. Galaxy power spectrum accuracy (CLAX-PT vs. CLASS-PT, $z = 0.38$).

Spectrum	Max error	Metric
$P_{mm}^{\rm real}$, $P_{gg}^{\rm real}$, $P_{gm}^{\rm real}$	0.31%	rel.
$P_{mm}^{(\ell=0)}$, $P_{gg}^{(\ell=0)}$	0.59%, 0.56%	rel.
$P_{mm}^{(\ell=2)}$, $P_{gg}^{(\ell=2)}$	0.70%, 0.89%	rel.
$P_{mm}^{(\ell=4)}$, $P_{gg}^{(\ell=4)}$	0.70%, 1.43%	abs/max

forward pass, so the effective speedup for $d = 6$ standard ΛCDM parameters is $2d/(1 + c_{\rm bwd}) \approx 3$–$4\times$, growing to $\sim 10\times$ for extended models with $d = 10$–20 parameters (neutrino masses, dark energy, bias parameters, counterterms). The key advantage is the asymptotic scaling: AD gradient cost is $O(1)$ in d, while finite differences scale as $O(d)$. Combined with the 34 s forward evaluation of `fit_cl`, this makes gradient-based sampling with exact first-principles theory predictions practical for full-shape galaxy clustering analysis.

5. Supervised vs. Unsupervised Development

5.1. Bug taxonomy

Over the full development of CLAX and CLAX-PT, we documented 40 bugs: 26 in the CMB pipeline and 14 in the PT module. We classify these into four types by their nature and whether autonomous resolution was possible.

Type I: Convention and unit bugs. Errors in translating CLASS conventions into JAX: wrong sign, missing numerical factor, incorrect unit conversion. *Agents resolved all Type I bugs autonomously* by reading the corresponding CLASS source code and correcting the factor.

Table 4. AD vs. finite-difference gradients for $P_{mm}(k = 0.1\ h/\mathrm{Mpc})$.

Parameter	$\lvert \mathrm{AD}/\mathrm{FD} - 1 \rvert$	Status
$\ln(10^{10} A_s)$	$< 0.01\%$	✓
n_s	$< 0.01\%$	✓
h	$< 0.5\%$	✓
ω_b	$< 1.0\%$	✓
$\omega_{\rm cdm}$	$< 0.5\%$	✓

Table 5. Performance summary.

Preset	GPU	Time	C_ℓ accuracy
`fit_cl`	V100-32GB	34 s	$< 1.5\%$ ($\ell \leq 500$)
`planck_cl`	H100-80GB	487 s	$< 0.2\%$ ($\ell \leq 2000$)

Type II: Algorithm implementation bugs. Errors in the numerical algorithm itself: wrong grid type, incorrect matrix storage format, incomplete angular kernel functions. *Agents resolved all Type II bugs autonomously*, typically by careful comparison of intermediate arrays against CLASS/CLASS-PT at specific (k, z) points.

Type III: Architectural bugs. The computational structure assumed by the implementation is fundamentally incapable of representing the correct physics. One instance occurred: the analytic Legendre projection architecture assumed isotropic BAO damping, making it unable to handle anisotropic damping $\Sigma^2_{\rm tot}(\mu)$. *This required human intervention.*

Type IV: Physics judgment failures. A numerical patch satisfies all oracle tests but encodes physically unmotivated behavior. One instance occurred: a scalar fudge factor $\alpha = 0.27$ passed all fiducial tests but has no physical derivation. *This required human intervention.*

The resolution breakdown is: Types I+II (38 bugs): 100% autonomous; Types III+IV (2 bugs): 0% autonomous. Overall, 38/40 bugs (95%) were resolved without human input.

5.2. Principles for human–agent collaboration

We distill three principles from the empirical data:

P1: Oracle testing verifies *what*, not *why*. An oracle compares numerical values at a reference parameter point. It catches implementation errors (wrong sign, wrong coefficient) but not physics errors (wrong model, numerically compensated). A fudge factor with zero degrees of freedom at the calibration point will pass every oracle test at that point. *Defense*: test across diverse parameter space; test intermediate quantities; build auxiliary tests that check structural properties (e.g., that $P_{\rm tree}(k, \mu)$ has the correct anisotropy as a function of μ), not just numerical values.

P2: Shared memory prevents re-exploration but not architectural loops. `CHANGELOG.md` records which algorithms were tried and failed, preventing agents from rediscovering dead ends. This was effective for Type I and II bugs. It did not prevent 33 sessions of exploration within the wrong architectural framework because all attempts were coherent within the (incorrect) model. *Defense*: when a bug persists after $N \approx 5$–10 sessions without progress, treat the lack of progress itself as evidence of an architectural problem and trigger human review.

P3: The irreducible human role is architectural and physical judgment. Agents excel at reading CLASS source code and transcribing equations faithfully; identifying which term disagrees via targeted comparison with oracle outputs; and adjusting coefficients and conventions. Agents fail at recognizing that an architectural assumption is inconsistent with the physics, and at detecting that a numerically adequate patch produces zero test failures but is not physics. *Detecting physics errors that pass all tests requires knowledge of what the physics should look like*, which is a form of human supervision that oracle testing cannot replace.

6. Related Work

Differentiable cosmological codes. DISCO-EB (Hahn et al., 2023) solves the linear Einstein–Boltzmann equations in JAX to compute the matter power spectrum but does not compute CMB spectra (C_ℓ) or galaxy clustering observables. ABCMB (Zhou et al., 2026) provides a complete CMB pipeline in JAX including lensing, massive neutrinos, and a state-of-the-art HyRex recombination treatment, but does not include perturbation theory for galaxy surveys. SymBoltz.jl (Sletmoen, 2024) provides a full Boltzmann solver in Julia with automatic differentiation but without GPU support. JAX-COSMO (Lemos et al., 2023) computes the nonlinear matter power spectrum but not the full Boltzmann hierarchy and not CMB spectra. CLAX is the first JAX code to provide full CMB spectra plus one-loop galaxy power spectra with exact automatic differentiation and GPU execution.

EFT power spectra and IR resummation. The EFT of large-scale structure (Perko et al., 2016; D'Amico et al., 2020) provides a systematic expansion for the galaxy power spectrum. CLASS-PT (Chudaykin et al., 2021) is the standard implementation; CLAX-PT reproduces it in differentiable JAX. IR resummation handles BAO peak broadening from long-wavelength displacements (Senatore & Zaldarriaga, 2015; Vlah et al., 2015).

Agent-assisted scientific software. Recent work has shown that LLM agents can complete extended scientific tasks autonomously (Carlini, 2026), but most evaluations focus on syntactic correctness (compilation, test passage) rather than physical validity. Our work identifies semantic correctness—the requirement that a passing numerical patch has physical basis, not just calibration-point accuracy—as the key challenge distinguishing scientific software from general code generation.

7. Conclusion

We have developed CLAX and CLAX-PT—fully differentiable JAX reimplementations of the CLASS and CLASS-PT Boltzmann codes—using oracle-guided agentic development. The codes achieve $< 0.2\%$ lensed CMB accuracy at $\ell \leq 2000$, $< 0.7\%$ galaxy power spectrum accuracy for monopole and quadrupole spectra, with automatic differentiation gradients verified to 0.15% and GPU execution at 34 s/step on V100. Empirically, AI agents resolved 95% of implementation bugs autonomously. The remaining 5% required human physics judgment: recognizing a structural mismatch between an architecture and its target physics, and detecting a numerically effective but physically unmotivated fudge factor.

These findings suggest a practical framework for AI-assisted scientific code development: agents operate autonomously in unsupervised mode for implementation work, but human scientists remain in the loop as architectural and physical reviewers, intervening proactively when oracle feedback stalls or when a numerical patch lacks physical derivation. The codes are open source at `https://github.com/smsharma/clax`.

References

Blas, D., Lesgourgues, J., and Tram, T. The cosmic linear anisotropy solving system (CLASS). Part II: Approximation schemes. *JCAP*, 2011(07):034, 2011. doi: 10.1088/1475-7516/2011/07/034.

Cabezas, A. et al. BlackJAX: Composable Bayesian inference in JAX. 2024.

Carlini, N. Coding agent builds a C compiler. Anthropic Technical Report, `http://anthropic.com/research/coding-agent-c-compiler`, 2026.

Chudaykin, A., Ivanov, M. M., Philcox, O. H. E., and Simonović, M. Nonlinear perturbation theory extension of the boltzmann code CLASS. *Phys. Rev. D*, 103:023507, 2021. doi: 10.1103/PhysRevD.103.023507.

Cranmer, K., Brehmer, J., and Louppe, G. The frontier of simulation-based inference. *Proc. Natl. Acad. Sci.*, 117: 30055–30062, 2020. doi: 10.1073/pnas.1912789117.

D'Amico, G. et al. The cosmological analysis of the SDSS/BOSS data from the effective field theory of large-scale structure. *JCAP*, 2020(05):005, 2020.

DESI Collaboration. DESI 2024 VI: Cosmological constraints from the measurements of baryon acoustic oscillations. *AJ*, 169(3):163, 2025.

Di Marzo, G. Méthodes de Rosenbrock d'ordre 5(4) adaptées aux systèmes différentiels-algébriques, 1993. Diploma Thesis, University of Geneva.

Fang, X., Blazek, J. A., McEwen, J. E., and Hirata, C. M. FAST-PT II: an algorithm to calculate convolution integrals of general tensor quantities in cosmological perturbation theory. *JCAP*, 2017(02):030, 2017.

Gabrié, M., Rotskoff, G. M., and Vanden-Eijnden, E. Adaptive monte carlo augmented with normalizing flows. *Proc. Natl. Acad. Sci.*, 119(10), 2022. doi: 10.1073/pnas.2109420119.

Hahn, O., List, F., and Porqueres, N. DISCO-DJ: Differentiable simulations for cosmology – Done with JAX. https://github.com/ohahn/DISCO-EB, 2023. arXiv:2311.03291.

Kidger, P. *On Neural Differential Equations*. PhD thesis, University of Oxford, 2021.

Lemos, P. et al. jax-cosmo: An end-to-end differentiable and GPU accelerated cosmology library. *The Open Journal of Astrophysics*, 6, 2023. doi: 10.21105/astro.2302.05163.

Lewis, A., Challinor, A., and Lasenby, A. Efficient computation of cosmic microwave background anisotropies in closed FRW models. *ApJ*, 538:473–476, 2000. doi: 10.1086/309179.

Neal, R. M. MCMC using Hamiltonian dynamics. In Brooks, S., Gelman, A., Jones, G. L., and Meng, X.-L. (eds.), *Handbook of Markov Chain Monte Carlo*, chapter 5. Chapman & Hall/CRC, 2011.

Perko, A., Senatore, L., Jennings, E., and Wechsler, R. H. Biased tracers in redshift space in the EFT of large-scale structure. 2016.

Phan, D., Pradhan, N., and Jankowiak, M. Composable effects for flexible and accelerated probabilistic programming in NumPyro. 2019.

Senatore, L. and Zaldarriaga, M. The IR-resummed effective field theory of large scale structures. *JCAP*, 2015(02): 013, 2015. doi: 10.1088/1475-7516/2015/02/013.

Sletmoen, H. SymBoltz.jl: A differentiable cosmological boltzmann solver in Julia. https://github.com/hersle/SymBoltz.jl, 2024.

Vlah, Z., White, M., and Aviles, A. A Vlasov-Poisson approach to the large-scale structure, with applications to the EFT. *JCAP*, 2015(09):014, 2015. doi: 10.1088/1475-7516/2015/09/014.

Zhou, Z., Giovanetti, C., and Liu, H. ABCMB: A Python+JAX package for the cosmic microwave background power spectrum. 2026. arXiv:2602.15104.